

COMMODORE USERS EUROPE · 26 SEPTEMBER 2026

Compunet **Reborn**

“Welcome back to the Live One!”

Tristan Greaves <tristan@extricate.org>

`connect.compunet.live`

Your presenter

- ▶ Favourite game: **Pool of Radiance**
- ▶ Computing journey:
 - ▶ Atari 400
 - ▶ Commodore 64C (then a disc drive...)
 - ▶ Amiga 500
 - ▶ Amiga 1200 (ending up in a **Power Tower**)
 - ▶ Windows / Linux / Mac....
 - ▶ C64 Ultimate + Apollo A6000 !



What was Compunet?

What was Compunet?

- ▶ A UK online service for the **Commodore 64**. 1984 to May 1993
- ▶ Compunet Teleservices Ltd — software by **Ariadne**
- ▶ You dialled in on a custom **1200/75 baud** modem: “the brick”
- ▶ **Custom ROM** with integrated off-line frame editor
- ▶ £0.80p call charges off-peak + Other subscription costs.
- ▶ Concept similar to **Quantum Link** (US / Canada)



Behold!

compunet		
1 *** COMPUNET ***		
100 WELCOME		PRICE
1 *** WELCOME ***	11	
USER GUIDE	D+	
NEWS	T1+	
WHAT'S NEW?	D+	
THE JUNGLE	T2+	
AMIGA ZONE	D+	
PARTYLINE	D+	
WHO IS ONLINE?	T1	
SUPPORT US	T1	
(F7)(F8)		
COMPUNET REBORN 1.3.0 RELEASED		
ADD CUSTOM HEADERS ON COMPUNET.LIVE		
UPLD	VOTE	HELP DIR SHOW BACK GOT

Three months later...



The challenge

What was lost, and what wasn't

LOST

- ▶ The servers and hard disks — **destroyed**
- ▶ No protocol documentation survived.



SURVIVED

- ▶ The **8K boot ROM** — it could be dumped to disk, so copies exist
- ▶ The **8K TERMINAL download** — users could 'CNSAVE' this to disk to save time and money with future connections. Saved this project!
- ▶ **SEQ files** — the page editor allowed saving to disk, so users kept their content (and could save others!)
- ▶ **Screenshots and magazine articles**
- ▶ D64 disk images (Scene collections)

The 'brick' problem

One of the goals of disassembly: Work out how the modem / protocol works.

- ▶ Hard-coded to the brick
- ▶ It is **not a Hayes modem**. No 'AT' command set, no standard anything
- ▶ So nothing off the shelf translates — **no emulator, no internet bridge**, no drop-in substitute

Off the table: **Building a dial-up service!**



Reborn

Starting patching.... It did not work out.

PATCH IT

- ▶ Add **Hayes-style modem support** into the existing 8K ROM
- ▶ Minimal disruption.
- ▶ **Too hacky** — bolting a different world onto IRQ-driven 1984 timing
- ▶ **And no room.** 8K is 8K, and it was already full

REFACTOR IT

- ▶ Disassembly all the way back to **source**
- ▶ A full **end-to-end compile chain**
- ▶ **Verified byte-identical** to the original ROM before changing a single line
- ▶ Slower to start. Easier to build upon.

Disassembled and commented: the VOTE handler (\$A29D)

```
; --- VOTE handler ($A29D) ---
; Saves screen state, clears type flag, calls vote subroutine, restores.
    JSR L_A79B                ; save screen state
    LDA $0286                ; save current text colour
    STA $C00A
    LSR $C028                ; clear vote type flag
    LDX #$01
    LDY #$08
    STX $2B
    STX $C00D
    STY $2C
    STY $C00E
    JSR L_B6DA                ; vote subroutine
    LDA $C00A                ; restore text colour
    STA $0286
    JMP L_A7A6                ; return to main loop
.byte $A9, $45, $8D, $00, $C1, $A0, $01, $20, $84, $A7, $20, .lobyte(L96D2), .hibyte(L96D2), $B0, $CA, $20
.byte $21, $81, $20, .lobyte(L96C0), .hibyte(L96C0), $A9, $0E, $8D, $20, $D0, $20, $86, $AC, $68, $68, $20
.byte $48, $81, $A9, $93, $4C, $D2, $FF, $20, $9B, $A7, $AD, $86, $02, $8D, $0A, $C0
.byte $A2, $23, $A0, $A3, $20, $42, $81, $A2, $06, $A0, $01, $A9, $00, $18, $20, $1B
```

Findings...

- 10 May The patched ROM reached the login screen, then stalled receiving the LINKING download
- 11 May The ROM ignored every reply until the **X.25 acknowledgement** was exactly right: 7 bytes
- 12 May The ROM's interrupt-driven packet code relied on the brick's timing, and a buffered serial port broke it. It became polling code, with an **NMI ring buffer** doing the brick's job
- 13-14 May Replies over 200 bytes crashed the C64: buffer overflow, an early end-of-stream, and sending that silently switched off receiving
- 13-16 May **VICE's** SwiftLink emulation strips bit 7 from bytes the C64 sends, breaking checksums and uploads

Findings... continued

23-24 May	C64 Ultimate : its modem bridge needed AT commands in ASCII rather than PETSCII, and lost bytes while dialling
27 May	VICE on Windows : leftover decompressor bytes on the 6502 stack caused CPU JAMs
31 May	Fixed delays between packets replaced by X.25 acknowledgements , so the C64 sets the pace
1 June	Two failures traced to the ROM itself: it zeroes the 6510's data-direction register mid-parse, and big directories overwrite the NMI vector at \$FFFA
Always	X.25 protocol preserved, client and server

It works

- ▶ Runs in **VICE**, the C64 emulator
- ▶ And on **real hardware**: the **C64 Ultimate**, or a C64 with a SwiftLink cartridge
- ▶ Boots from an **8K cartridge image**, like the brick
- ▶ Recorded at 0.1 beta, 18 May

```
**** COMMODORE 64 BASIC V2 ****  
64K RAM SYSTEM 38911 BASIC BYTES FREE  
COMPUNET REBORN 0.1BETA  
READY.
```

X.25 in action: “show page 7”

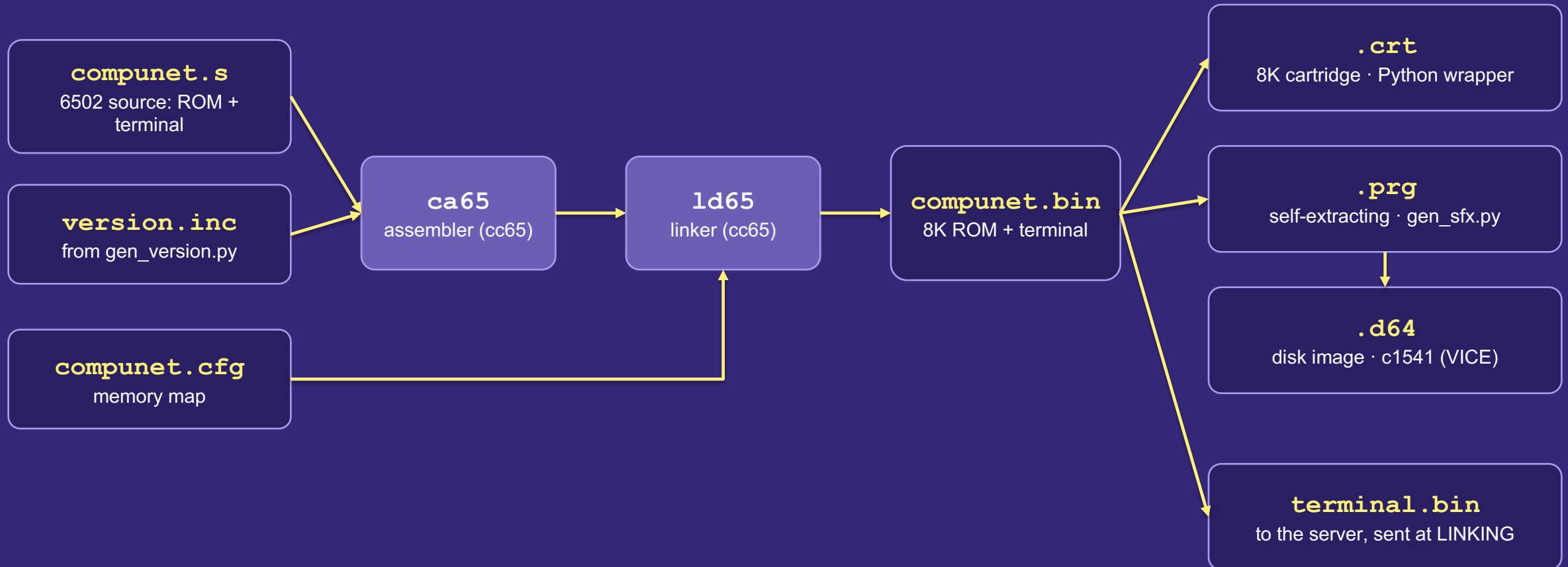


Stop-and-wait

One packet in flight.
The server waits for
each ACK, so the
C64 sets the pace.

Every packet: framed by \$01...\$02, sequence-numbered and CRC-checked. Framing bytes inside the packet are escaped.

The C64 build chain



Built twice: standard, and **live** (auto-connect, `ca65 -D AUTO_CONNECT`). One **make** builds everything and copies the downloads to the website.

The server

- ▶ **Python 3.12** with `asyncio`: one process serves every connection
- ▶ Built on **`aiohttp`**, with `markdown` the only other dependency
- ▶ **No database**: pages are SEQ files and JSON on disk
- ▶ One server for every client: C64, Amiga, terminal and browser
- ▶ Runs in **Docker**, with the website (Flask) in a second container

Content on disk: folders and JSON

ON DISK

```
root/
├── root.json           # the top menu
├── header.seq
├── news/
│   ├── directory.json
│   └── frame-1.seq
├── jungle/
│   ├── directory.json
│   ├── header.seq
│   ├── frame-1.seq
│   └── ash-and-dave/
│       ├── directory.json
│       └── frame-1.seq
└── partyline/
    ├── directory.json
    └── partyline.prg
```

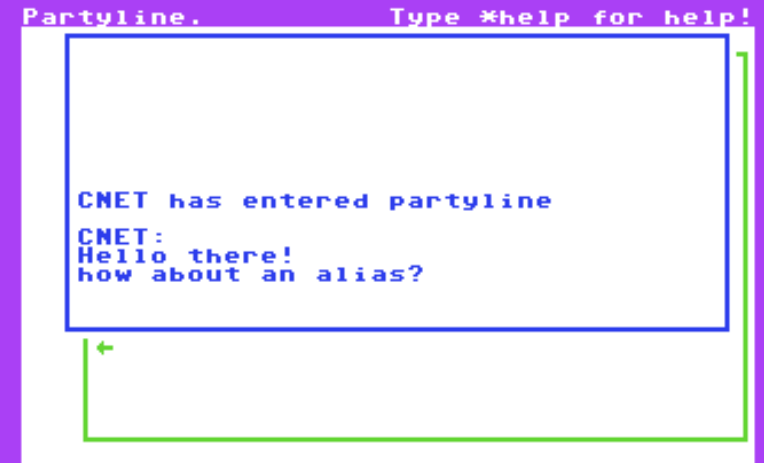
ONE ENTRY IN `root.json`

```
{
  "page_num": 600,
  "title": "JUNGLE",
  "type": "T",
  "author": "JUNGLE",
  "price": 0,
  "life": 105,
  "keyword": "JUNGLE",
  "frames": ["frame-1.seq"],
  "directory": "jungle/directory.json"
}
```

Every page is a folder: its frames are SEQ files, and a `directory.json` lists the pages beneath it. Files are re-read on every request, so edits go live without a restart.

Partyline

- ▶ The live multi-user chat.
- ▶ Presumed **lost in time**.
- ▶ Then an **offline demo** of Partyline turned up.
- ▶ Built to show the thing working without dialling in.
- ▶ 'Look and feel' and a selection of commands could be extracted.



The screenshot shows a terminal window with a purple background. At the top, the text "Partyline." is on the left and "Type *help for help!" is on the right. A large white rectangular box with a blue border contains the text: "CNET has entered partyline", "CNET:", "Hello there!", and "how about an alias?". Below this box is a green rectangular input area with a green cursor arrow pointing to the left.

```
Partyline.                Type *help for help!  
  
CNET has entered partyline  
CNET:  
Hello there!  
how about an alias?  
  
←
```

Same lesson, second time

- Amiga client presumed lost – I couldn't find it anywhere! (Aminet, Internet searches!)
- July 2026: A real **1989 Amiga Compunet client** surfaces on a PD disk (17-bit)
- Straight to disassembly and byte-exact reconstruction.
- “It looks squashed!” – yep, this is authentic!

C64 client ~1 month

Amiga client ~5 days



Amiga: 68000 disassembly → reconstructed C

ORIGINAL, 1989

```
10a29a moveq    #$2, d0
10a29c move.l   d0, $70(a4)
10a2a0 pea.l    $1404(a4)
10a2a4 jsr      $10a52e(pc)
10a2a8 addq.w    #$4, a7
10a2aa pea.l    $43.w
10a2ae pea.l    $1.w
10a2b2 move.l   d0, -(a7)
10a2b4 pea.l    $1402(a4)
10a2b8 jsr      $10a5ca(pc)
10a2bc lea.l    $10(a7), a7
10a2c0 pea.l    $321a(a4)
10a2c4 jsr      $10a51c(pc)
10a2c8 addq.w    #$4, a7
10a2ca cmpi.b   #$40, d0
10a2ce beq.b    $10a2d4
10a2d0 moveq    #$0, d0
10a2d2 bra.b    $10a2e0
10a2d4 move.l   $7c(a4), -(a7)
10a2d8 bsr.w     $109a5e
10a2dc addq.w    #$4, a7
10a2de moveq    #$1, d0
10a2e0 rts
```

RECONSTRUCTED C

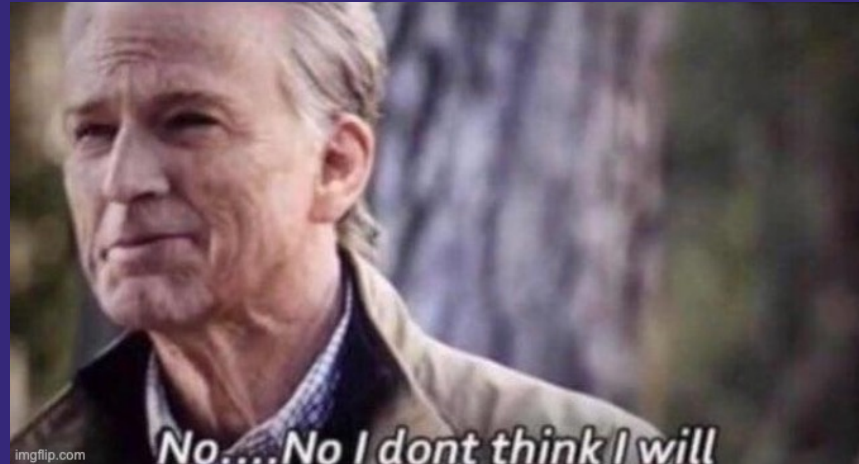
```
LONG back_dir(void)
{
    g_state = STATE_GOTO;                /* recon 0x10a29c: g_state = 2 */
    serial_write("B", 1, 1, TOKEN_COM);  /* recon 0x10a2b8: send "B" */
    if (serial_io_c(g_ack_text) != ACK_OK) /* recon 0x10a2ca: ack != '@' */
        return 0;
    directory_refresh(g_dir_page);        /* recon 0x10a2d8 */
    return 1;
}
```

The **Back** button: send "B" as a COM packet, wait for the '@' acknowledgement, then reload the directory.

Every line of C cites the address in the original program it was reconstructed from.

Deliberately switched off: run-on-arrival code

- ▶ The 1989 Amiga client has an “**action**” page type: it downloads a program to RAM:temp and **runs it immediately**
- ▶ Its only check is that the file is marked as an Amiga program
- ▶ So the server refuses it: selecting one shows **NOT AVAILABLE**, and uploads of that type are rejected on every path



In 1989 this was a feature. Today remote code execution is **BAD!**

Taking it up a level

Third time's a charm

C64	Reverse-engineered the hard way. One month
Amiga	Same method, applied from day one. Five days
Third time	Stop reverse-engineering. Write it down.

A formal, platform-agnostic **specification**: normative, testable, and independent of any one machine.

Which means a client for **anything** is now a reading exercise, not an archaeology project.

§2 — Transport

Every packet is delimited on the wire by two marker bytes:

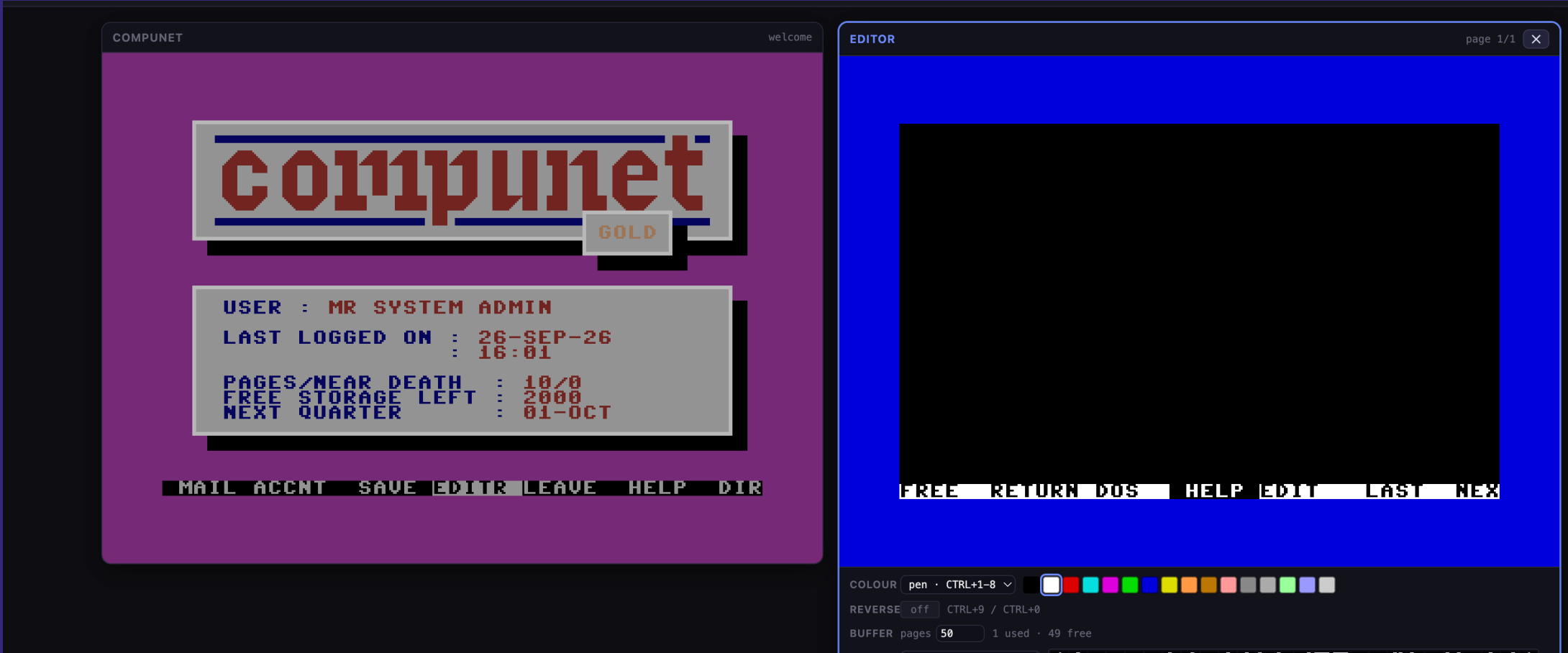
```
$01 <byte-stuffed content> $02
```

A receiver **MUST** locate packets by scanning for \$01, then reading up to the next \$02. Any bytes received before a \$01 **MUST** be discarded.

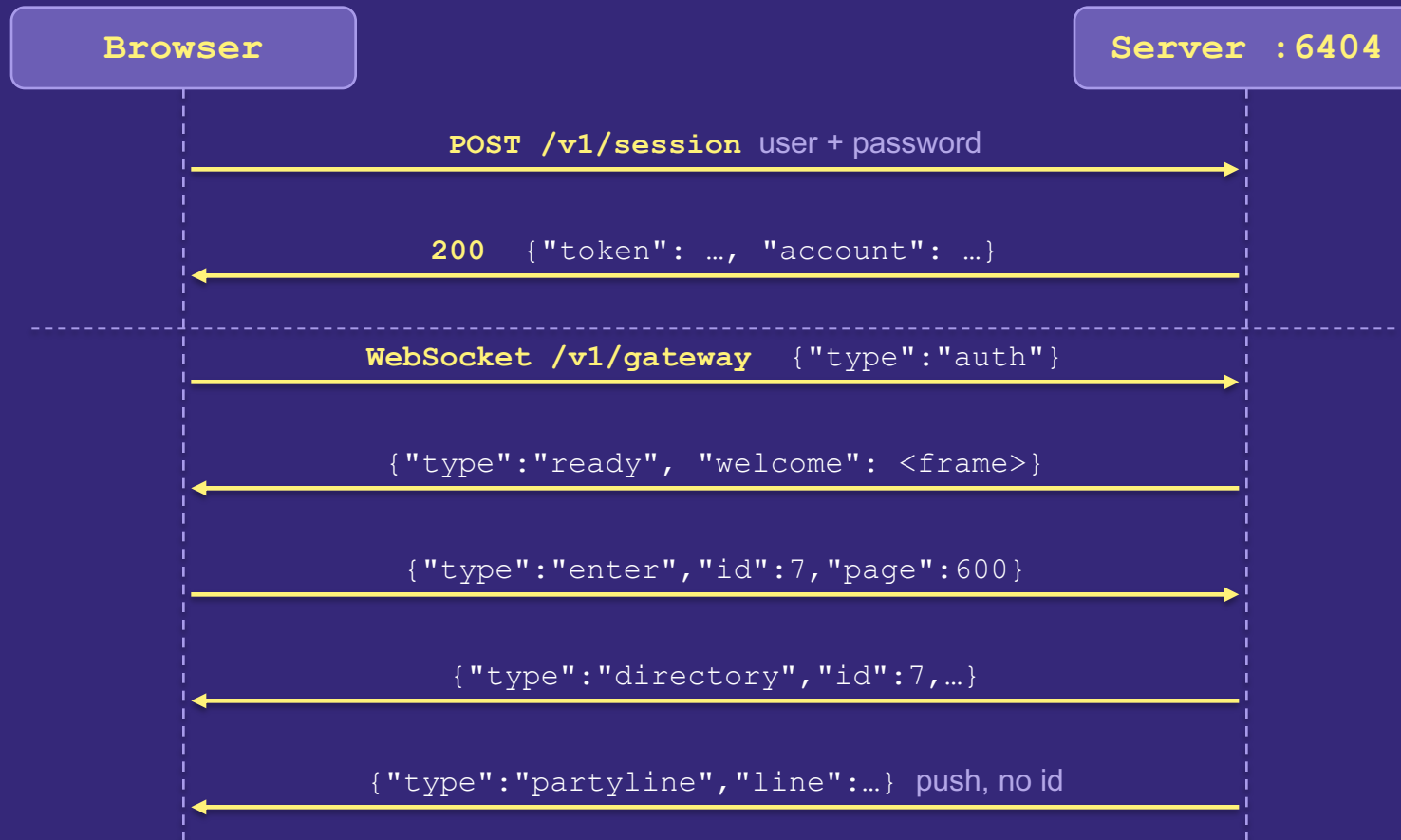
The three byte values \$01, \$02 and \$03 are reserved for framing and **MUST NOT** appear literally inside packet content on the wire.

Content byte	On the wire	Note
\$00	\$00	sent as-is
\$01	\$03 \$21	escaped (would otherwise be a start marker)
\$02	\$03 \$22	escaped (would otherwise be an end marker)
\$03	\$03 \$23	escaped (the escape byte itself)
\$04-\$FF	unchanged	sent as-is

Spec-built web client



The web client: JSON over WebSocket



Same service, different wire

Same commands, same content, same session code as the X.25 path

Frames arrive as a 40×24 grid of cells, already decoded: the browser just paints them

Directories arrive as lists, so moving the selection needs no round trip

Replies echo the request's id; server pushes like Partyline carry none

HTTPS for the login, then one WebSocket for the whole session. `api_binding.py` is a translator onto the same server core, not a second implementation.

Summing up...

Running costs, then and now

THEN: MAINFRAME

£50,000

per month, for about 50 users
(allegedly)

NOW: COMPUNET REBORN

£10

per month, for thousands of users

What exists now

- ▶ C64 client — 6502 assembly, 8K ROM + downloaded terminal, as it was
- ▶ Amiga client — native, reconstructed from the 1989 original
- ▶ Browser and desktop clients — zero setup, nothing to install
- ▶ A formal, platform-agnostic protocol spec — and nine clean-room clients built from it

From one dead service to a specification anyone can build against.

Join us.

`compunet.live`

Offline since 1993...

THANK YOU

Questions?

Try it `connect.compunet.live`

Source `github.com/tgreaves/compunet-reborn`

Historical material contributed by Graeme Norgate, Frank at Games That Weren't, Neil Shumsky, Mark Wilson, Richard Hawkins, Charles Headey and Zildac.